WebToSpeech: Final Report
By: Chandler Boone, Brady Byrd, Danny Lee, John Hurley

Table of Contents

Introduction
Scope
Timeline
UI Design
Sign-in Page
Dashboard Page
Document Page
Profile Page
Technology
Design
Database
Design Philosophies
Deployment
Known Issues

Introduction

WebToSpeech is a web-based document reader that lets users upload their documents and have them read aloud via text-to-speech. Users will also be able to create collections to organize their documents, keeping related files in one place that can be accessed and viewed within the app. WebToSpeech provides users with a more accessible reading experience for those with reading disabilities, and is also a general platform for any user to have their documents read aloud for entertainment, education, or other purposes.

Scope

The planned project scope is to make WebToSpeech a text-to-speech application and file organizer compatible with both desktop and mobile. The finished product should be able to perform text-to-speech on text, PDFs, and ePub files. When viewing a document, there should be many display options, such as page themes, text size, and font. The text-to-speech should highlight text, turn pages, start at any chosen position, change speed, and start/pause. The file organizer on the dashboard should allow users to upload files, delete files, search by title, create/update/delete collections and books, and filter by genre and file type.

Many of these features exist within the project in some form, but not all are complete or fully realized. File uploading currently supports only .txt files; text-to-speech cannot be resumed from a specific point; there are no options to change the reading voice; search filtering does not work; and more.

Timeline

The first month of the project was spent planning. During this phase, the group spent most of our time creating our proposal, requirements document, and presentation. The proposal was a document we had written outlining the project plan. This included the project concept, the features we intended to include, and the technology we planned to use to create it. While writing the project documentation, we created a prototype to test functionality as we worked on a more finished version of our designs. We also created concept art in Photoshop to form the visual identity and conceptual layouts for all of our pages. This phase lasted for the first month of development. After this phase ended, our group worked to integrate our technologies and implement the functionality and features outlined in our requirements document. This involved creating finalized HTML layouts, finalizing core features such as text-to-speech integration and collection creation, and bug testing to prepare the project for presentation. Development progressed smoothly until the last two weeks before the Innovation Experience, when work was split between the codebase and the poster to be presented alongside our project.

UI Design

When designing the User Interface for WebToSpeech, our priority was on a clean design that anyone could pick up and use. As such, it was essential that the elements have their purpose apparent and that nothing be overly cluttered. For our color scheme, we chose a white backdrop with maroon accents. This was done because the white is clean and easy, while the maroon accents evoke a feeling of both passion and sophistication that fit the website well. For our logo font, we chose Courgette, which also evokes a feeling of sophistication and simplicity. Before we began creating any of our webpages, we would create mock-ups of their layouts. During the prototyping period, this was done on paper to get a quick feel for how everything should look. As the project progressed, mock-ups were created in Photoshop to refine visual design elements and foster brand cohesion. To integrate these designs, we uploaded the finished mock-ups into Figma and had the “Make” project type AI generate React-based layouts utilizing the same visual themes. This allowed us to create the layouts of our pages relatively quickly, though we still had to spend time integrating our technologies and making the UI functional.

Sign-in Page

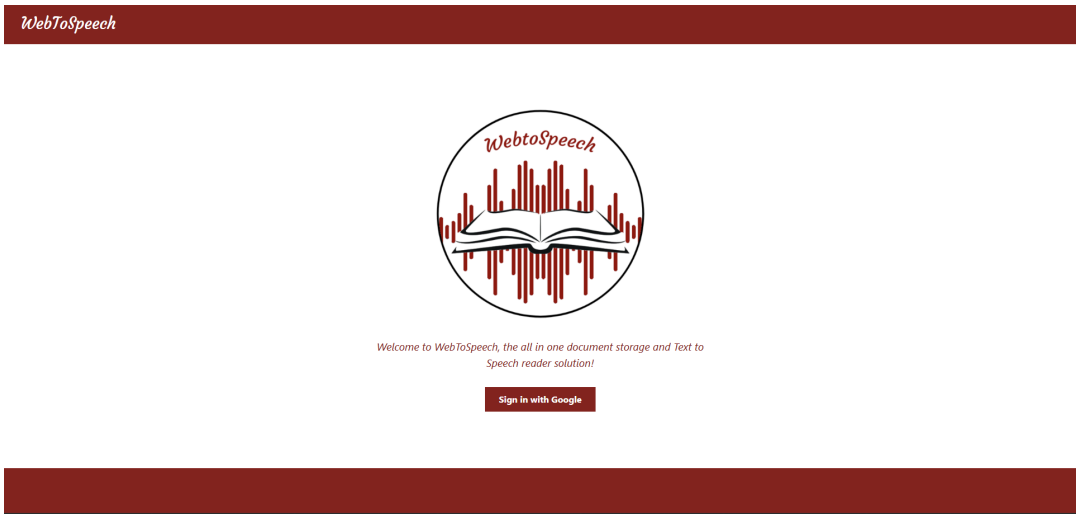


Image 1. WebToSpeech Sign-In Page

The sign-in page features the WebToSpeech logo in the middle, a circle containing an open book, with sound waves as a backdrop. The top and bottom of the page have the maroon trim. A brief welcome message is displayed below, followed by a button that allows the user to sign in with Google OAuth. Upon clicking the button, the default Google sign-in will be displayed, and once the user signs in, they will automatically be redirected to the Dashboard.

Dashboard Page

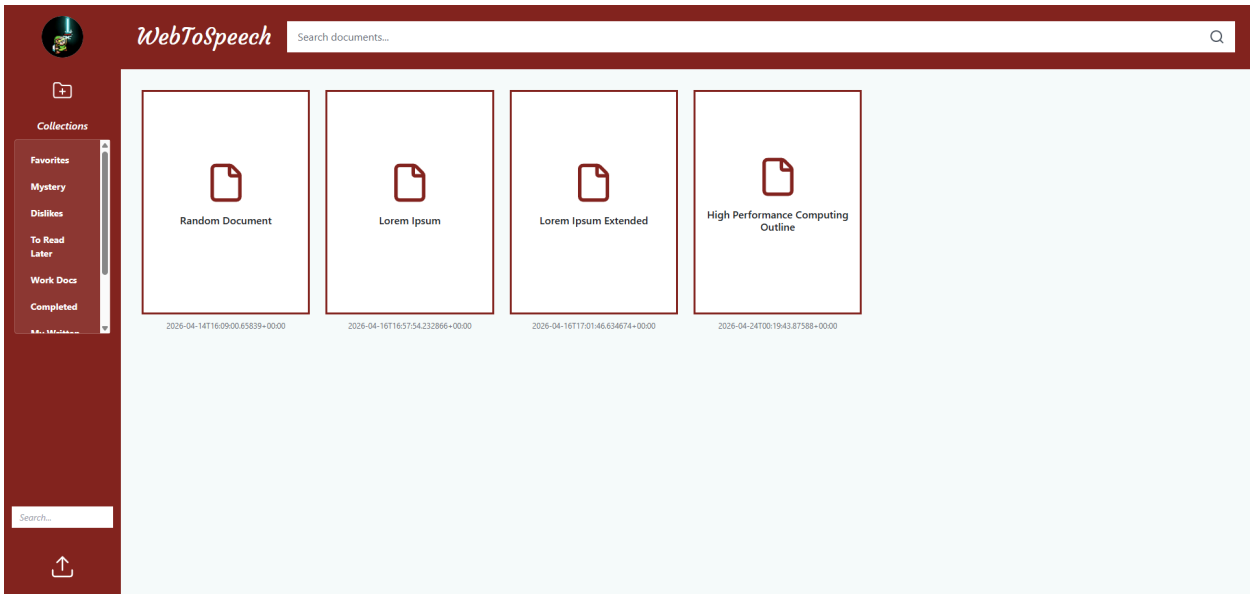
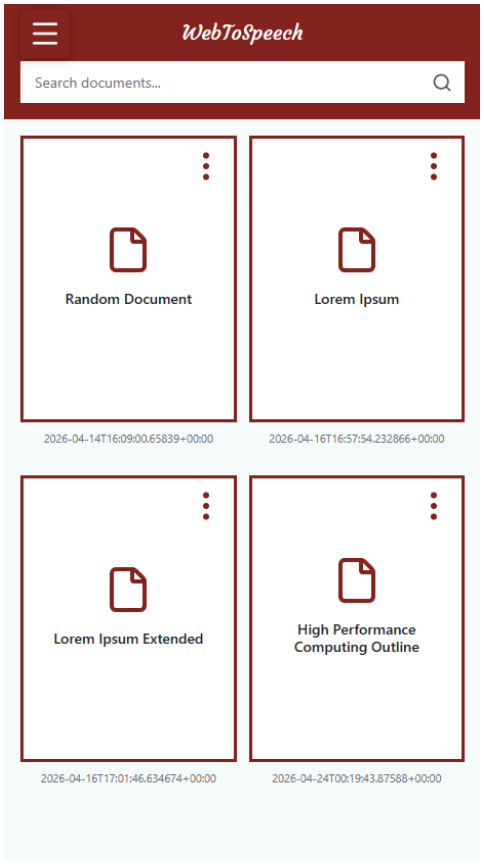
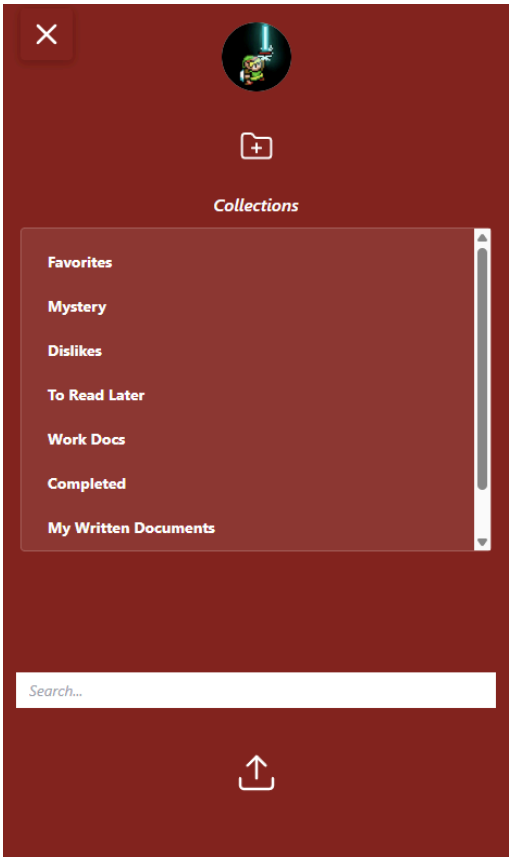


Image 2. WebToSpeech Dashboard Page (Desktop View)



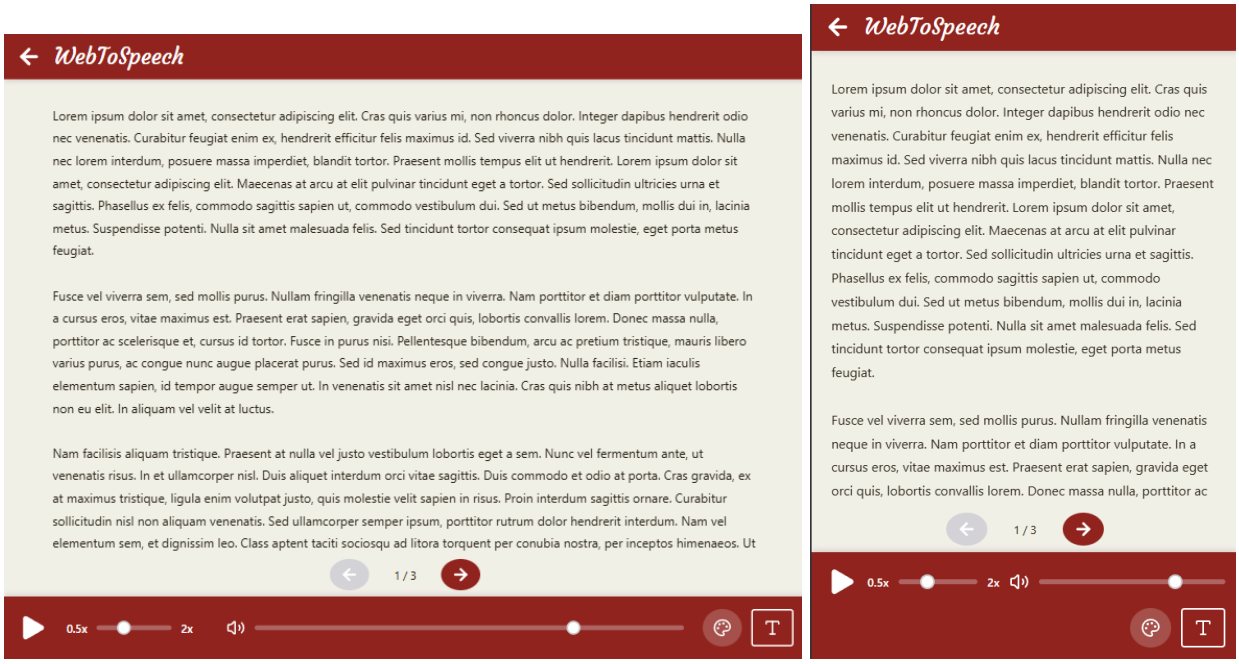


Images 3 and 4. WebToSpeech Dashboard Page (Mobile View)

The Dashboard page features three main sections. The header, the sidebar, and the document display area. Both the header and sidebar are maroon to provide contrast with the document display area. The header features the WebToSpeech title and a search bar. The sidebar features a profile button that takes the user to the profile page, an add collection button that opens a modal prompting the user to name the new collection, and a scrollable list of the user's collections. At the bottom of the sidebar is a search bar for collections and an upload button to upload new documents. When displayed on a mobile device, the sidebar is hidden and replaced with a menu button on the top left of the screen. Clicking the menu button displays the sidebar options, allowing the user to use the full mobile functionality while maintaining an organized viewing experience. The document display area shows cards for each of the user's uploaded documents, arranged in a grid that adapts to the user's display size. The user can filter the shown documents by searching for the document's name and pressing Enter. When the search bar is empty, the search is automatically retriggered to show all document cards again. The document cards have a three-dot menu that will display options when clicked. These options include editing the document details, adding the document to a collection, removing the document from a collection if being shown from

inside one, or deleting the document entirely. This menu is hidden until hovered over on desktop view, and always visible on mobile.

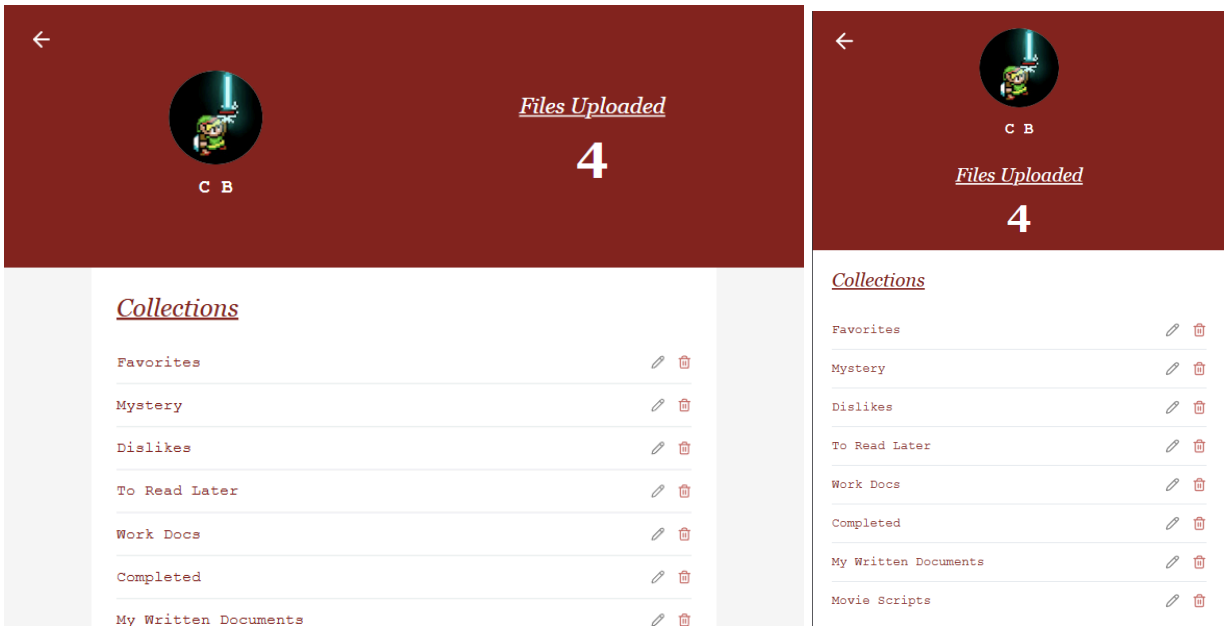
Document Page



Images 5 and 6. WebToSpeech Document Page (Desktop and Mobile View)

The document page includes a button to return to the dashboard in the top-left corner. At the bottom of the page, there are controls for the text-to-speech, including pause, speed, and volume. Text-to-speech reading can also be paused or resumed with the spacebar. The speed and volume sliders both immediately change the text-to-speech output, reinitializing the reader to continue from the same position with the new speed or volume applied. There are also options to change the color scheme of this page to white, sepia, or black, which will adjust the background and text colors accordingly. Finally, at the bottom is a button that lets the user change the page font size. When the user clicks the font size button, a modal appears with a slider that lets them select a font size between 12px and 24px, with 16px as the default. In the center of the page, the document's text is displayed. This area is scrollable until it reaches the bottom of the page, at which point the user can go to the next page using the arrow keys at the bottom. If the text-to-speech is active, the page will also be turned automatically once the reader finishes the last word on that page. The layout of this page is mostly identical on both desktop and mobile devices, though the controls bar wraps the theme and font-size buttons to provide more control for the speed and volume sliders.

Profile Page



Images 7 and 8. WebToSpeech Profile Page (Desktop and Mobile View)

The profile page shows the user's Google profile picture and username, followed by the number of files they have currently uploaded to WebToSpeech. These are displayed in a row when viewed on a desktop or displayed in a column when viewed on a mobile device. Below this is a list of the user's collections, and buttons to edit or delete each collection. Clicking the edit button, portrayed with the pencil icon, displays a prompt for the user to set the collection's new name, which will update it across all pages. Clicking the delete button, represented by a trashcan icon, displays a confirmation dialogue prompting the user to confirm the deletion of the collection. At the bottom of this page is the logout button, which will take the user back to the sign-in page.

Technology

The most prominent technology in our project is React. React had a decent learning curve for the group, as none of us had really utilized it before. At first, it was a bit overwhelming to figure out, but after we understood how the components worked, it became pretty seamless and helped keep the code clean. This had a significant impact on development, as it gave us reusable components, organized HTML and JavaScript, and influenced how we navigated each page.

To bring our 4-page design into React, we used React Router to navigate between the pages. It took a moment to set it up in the project, but once implemented, it made navigation seamless and hasn't needed any tinkering since.

We also used Lucide React for easy SVG implementation, which let us create clean-looking UI elements without importing images or drawing them ourselves. Lucide React is quite easy to use: we just import the SVG graphics as needed and the component for each graphic when needed.

Figma had a massive impact on our design workflow for the project. We were looking for a way to streamline the process of turning our page concepts into a functional website, and Figma provided the perfect solution. We were able to use their "Make" project type, providing folder structures and concept images, and it would then seamlessly generate React-based .jsx files for us to bring into the project. This allowed us to focus more on the page functionality and backend, rather than worrying about CSS tweaks and getting the layouts just right. Figma was also a huge help in creating functional mobile versions of our pages, generating some of the reactive CSS that handles the shifting layouts. This sped up the HTML, CSS, and layout work quite significantly while being easy to use.

For our Database, we ended up using Supabase. Supabase had a steep learning curve, with many intricacies to figure out during implementation. Finding documentation proved difficult due to compatibility with previous versions of the framework. Supabase was instrumental to the page's functionality, as we need to store the uploaded books, collections, and other user information.

Google OAuth, in conjunction with Supabase for user data storage, was used for our sign-in functionality. This allowed us to provide a simple, secure way for users to access the website. We were also able to use the Google account information to display their username and profile picture, rather than managing that information ourselves.

For the text-to-speech functionality on our webpage, we used the Web Speech API. We had initially settled on using a JavaScript TTS API called "Talkify", but after running into concerns with token limits based on Talkify's free usage plans, we ended up pivoting to the browser-based WebSpeech API so that any concern of token limits was completely nullified. The WebSpeech API itself wasn't super hard to integrate, though getting it to work as intended with our pagination systems and text highlighting posed some minor challenges.

For deployment, we ended up using Vercel. Vercel offered a free-to-use plan that provided us with an accessible URL and was relatively easy to set up. With the GitHub repository linked to the Vercel project, all changes we make to the main branch are reflected in the deployed version. It also worked well with our other technologies, thus providing the most reasonable deployment option for us.

Design

- Classes
  Book(id: String, title: String, author: String, genre: Array, fileType: String, uploadDate: String, path: String): Book
    - addGenre(genre: String): void
    - hasGenre(slug: String): boolean
  Collection(): Collection
    - getCollection(): Collection
    - addBook(book: String): void
    - deleteBook(bookID: String): void
    - checkForBook(book: String): boolean
    - getBook(findTitle: String): Book
    - filterByGenre(slug: String): Array
  UseSpeech(content: String): Object
    - isPlaying: boolean
    - speed: Number
    - volume: Number
    - highlightIndex:{start: Number, length: Number}
    - handlePlayPause(): void
    - handleSpeedChange(newSpeed: Number): void
    - handleVolumeChange(newVolume: Number): void
    - stop: void
  UsePagination
    - currentPage: Number
    - totalPages: Number
    - pageContent: String
    - goToNext: void
    - goToPrev: void
    - goToPage(n: Number): void

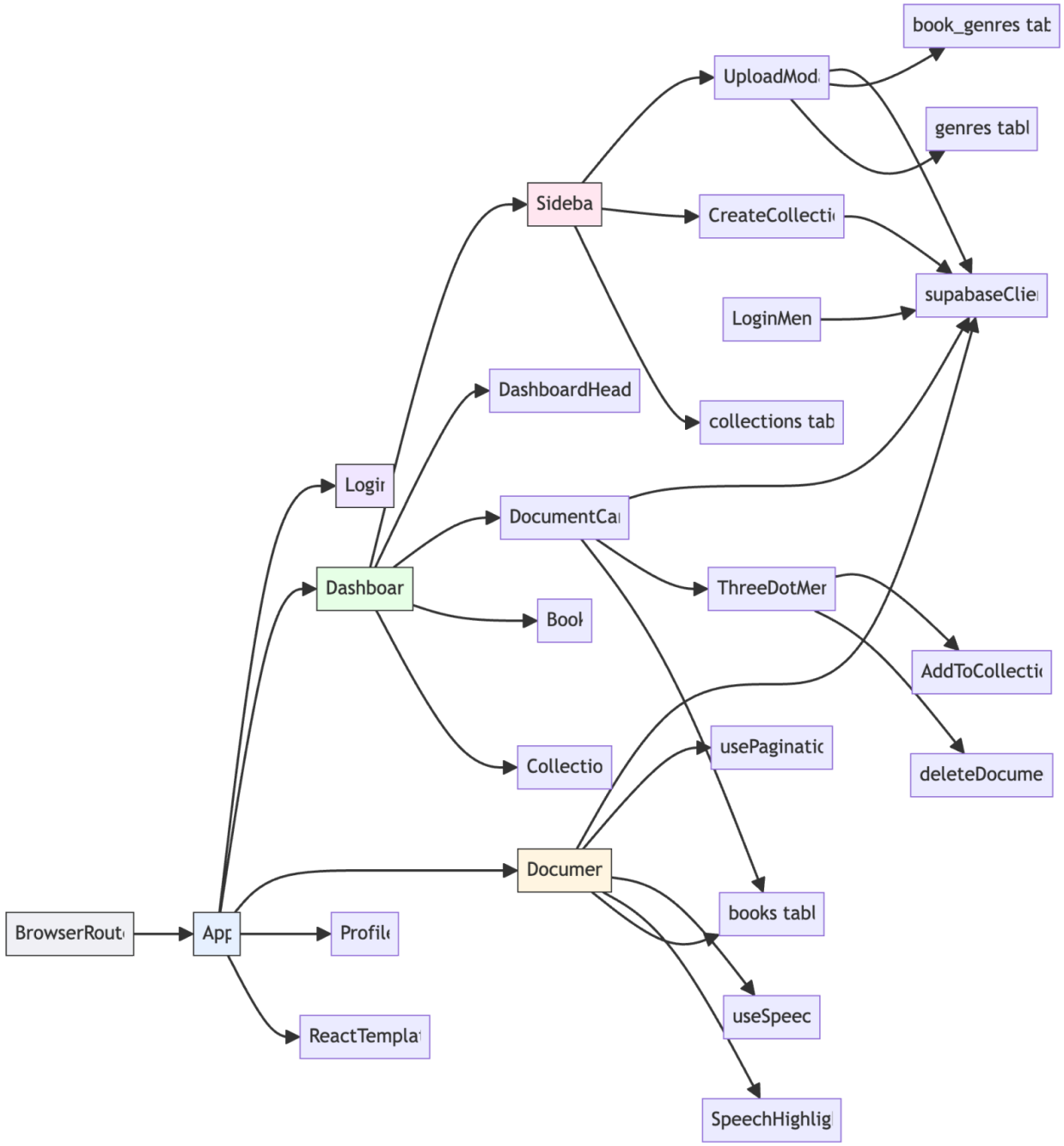


Image 8. WebToSpeech Project Component Tree

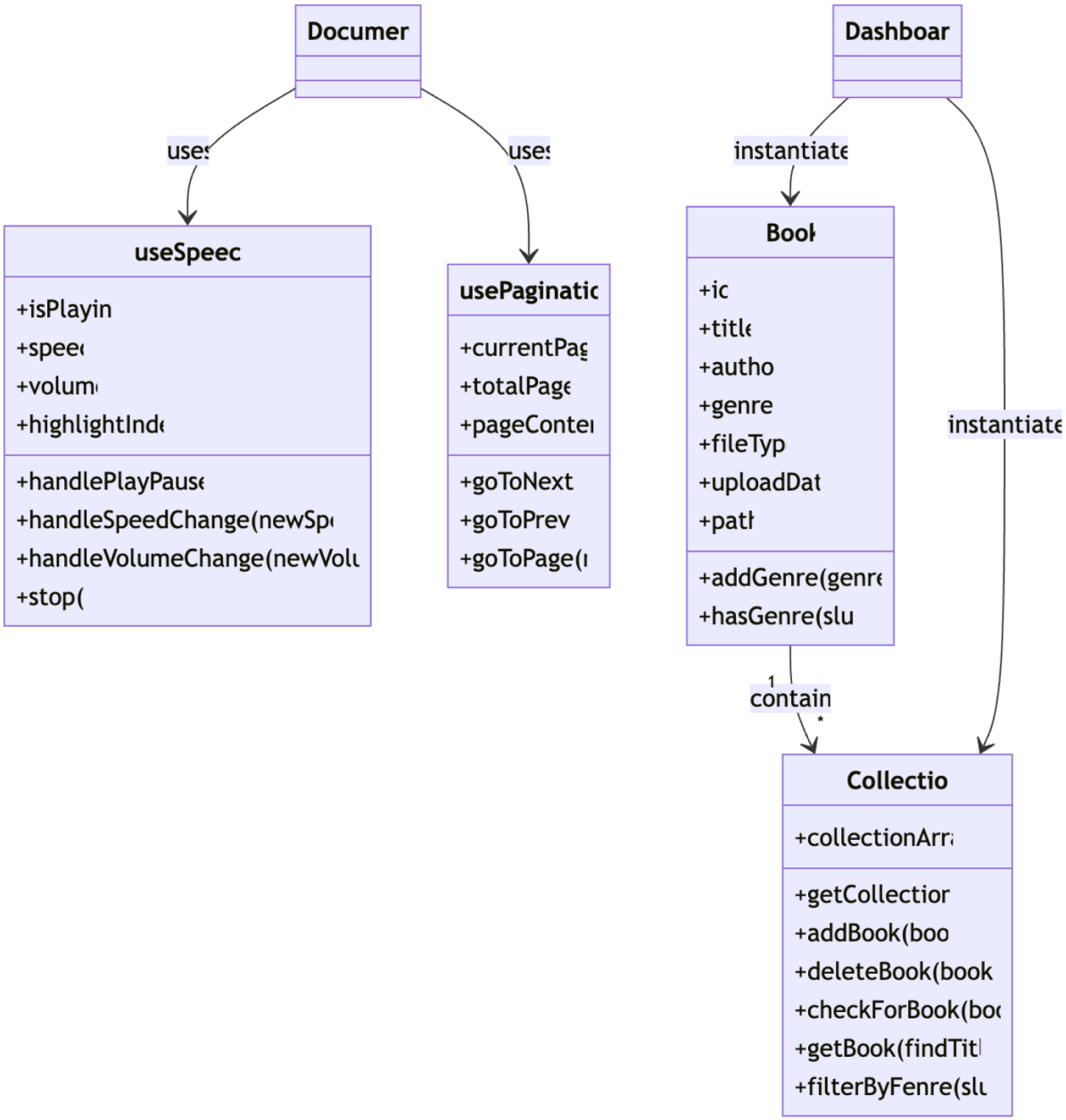


Image 9. WebToSpeech Project UML Class Diagram

`App.js` defines routes and maps URLs to page components. This component gets loaded by public/index.html and uses the URL to display the WebToSpeech pages to the user. This file was generated by the React project and was only altered to suit our routing needs.

`Dashboard.jsx` renders the main library view and builds a `Collection` from fetched `Book` records. This is the central page for the WebToSpeech project, being the first thing the user sees once they have signed in, and is used to reach both the Profile and Document pages.

`Document.js` is the reader page and connects text playback to `useSpeech` and pagination via `usePagination`. This page loads text data from a file and lets you view the document or listen to it using text-to-speech. The user can use the controls at the bottom of the page to navigate between pages, start and pause the text-to-speech reading, adjust the reading volume and speed, change the page color scheme, and adjust the font size and text size.

‘Profile.jsx’ handles the user profile page where a user can view their account info, manage collections, and log out. Supabase is used for authentication and database operations on this page, and cached user data is loaded from sessionStorage to display the username, profile picture, and number of uploaded files. Collections have edit and delete options that update the Supabase table for collections when used. The profile also has a link back to the dashboard page, or a way back to the sign-in page when the sign-out button is pressed.

‘LoginMenu.jsx’ handles user authentication by providing a simple sign-in option using Google OAuth through Supabase. It defines a function called signInWithGoogle that triggers the Supabase authentication flow. This redirects the user to the Google login page and brings them back to the app after signing in. If there is an error during login, it is logged to the console. Signing in with Google is the only option provided to users to minimize security risks, create a reliable way to manage accounts, and provide ease of access.

`Sidebar.jsx` manages uploads and collection creation with modal components. Visible from Dashboard.jsx, this component is used to navigate to the Profile page, display the user's collections, allow them to search their collections, and display a prompt for the user to upload their documents.

`DocumentCard.jsx` handles document navigation and in-card menu actions for editing, deleting, and adding to collections. These are displayed in a grid within the Dashboard.jsx page to allow the user to load a document and navigate to Document.js. On a desktop, hovering over a DocumentCard element shows a button that can be clicked to show the menu actions. On mobile, the CSS is altered so that the button is always visible.

‘DashboardHeader.jsx handles the top navigation bar of the WebToSpeech dashboard and provides a search feature that lets users filter documents. It keeps track of the user’s input using a state variable, query, and sends that value to the parent component via the outputSearch function whenever the user presses Enter, clicks the search icon, or clears the input.

‘CreateCollection.jsx’ ensures the user is logged in, then opens a modal menu prompting the user to enter a name for a new collection. The user's input is then sent to Supabase, and a new row is added to the collection table with the given name. This also handles all error cases in case something goes wrong during the upload process and displays a corresponding error message.

‘Upload.jsx’ is a modal that allows a logged-in user to upload a file (currently, only text files are fully operational), along with the document's title, author, and genre, to add to their page. Multiple genres can be selected at once, and when the create button is pressed, the data is sent to both the books table and the book_genres table in Supabase.

‘SpeechHighlight.jsx’ renders a block of text and highlights a specific portion of it based on a given character range. Takes in content, a highlightIndex object containing a start position and length, and a font size to control how large the text appears. Text is split into three parts: before the highlighted section, the highlighted section, and after the highlighted section.

‘DashboardHeader.css’ styles the dashboard header, adding the app title and search bar. Designed to be responsive for different screen sizes. The header is set up as a flex container with a red background, white text, and the Google Font ‘Courgette’ as the standard font for the WebToSpeech title. The search bar stretches to fill available space and has a search icon placed at the far left.

‘DocumentCard.css’ stylizes the document cards on the dashboard and the modal and drop-down interactions they support. The document Card is a white tile with a 3:4 aspect ratio and red accents. When hovered over, a faint shadow is applied to make it feel clickable, and the three-dot menu fades in on desktop. The three-dot menu is always visible on the mobile layout.

‘SideBar.css’ stylizes the sidebar layout, modal popups, and genre selection UI for the app. The sidebar is a vertical panel that appears with a fixed width on desktop, and as a full-width layout on mobile. Contains a button that leads to the profile page, a list of all collections uploaded by the user, and an upload button that opens a modal for uploading.

`supabaseClient.js` centralizes Supabase initialization and is imported by most data-access components. This script pulls environment variables into a Supabase “client” object that is used to make queries for documents and collections, handle document uploads, handle updates to document details and collection names, and load document information when entering the Document page.

Database

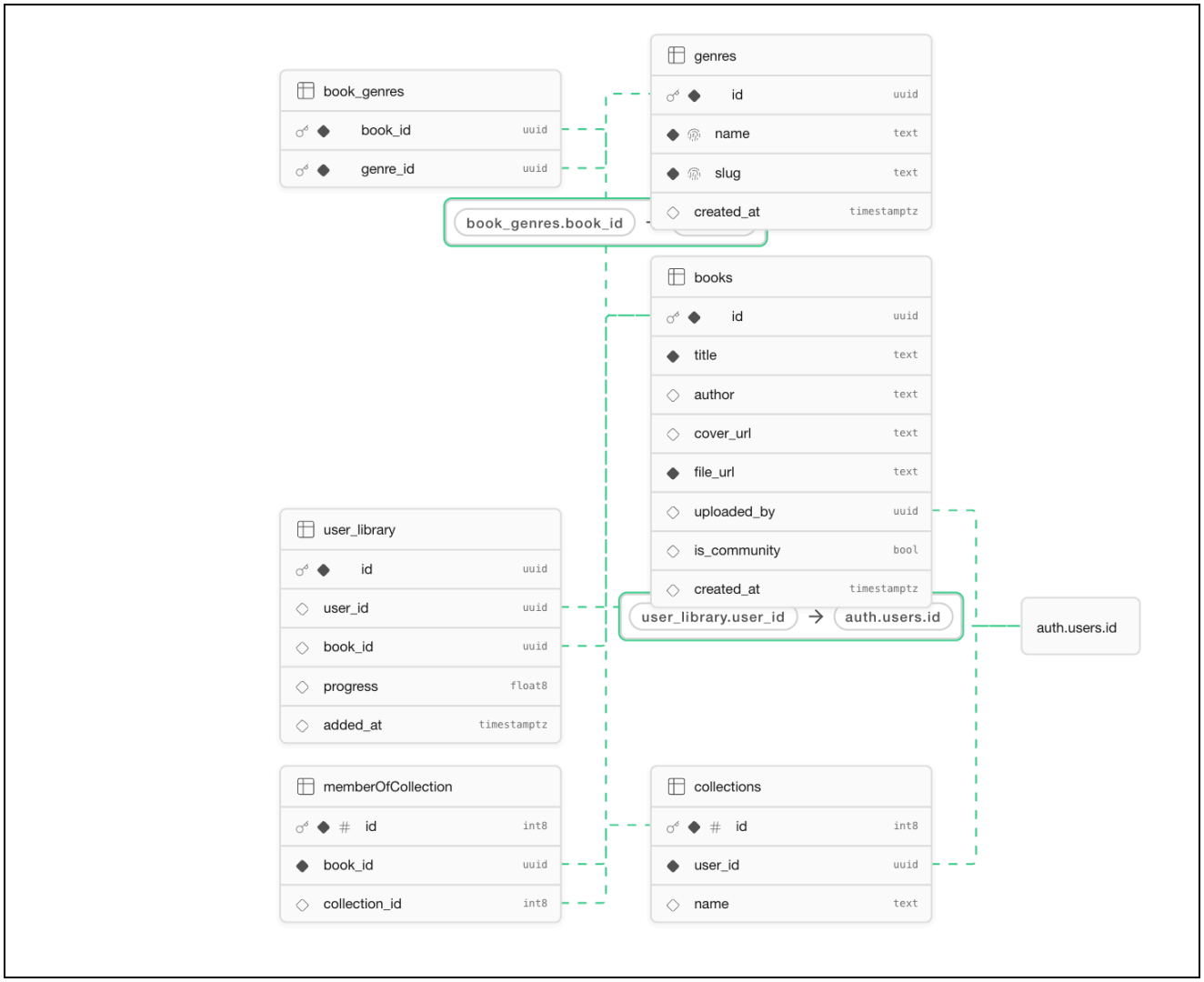


Image 10. WebToSpeech Supabase ERD

* Google Cloud: Google Auth Platform - Which connects the Google authentication to our Supabase project.

Design Philosophies

Some of the design philosophies our group implemented while developing our project included the single-responsibility principle, modularity, and mobile-first

development. The single responsibility states that a module should be responsible for only one, and only one, actor. This means a class should have only one reason to change. Our group implemented this design philosophy in our codebase when creating helper classes, such as supabaseClient.js, and when constructing the pages for our website. One page that we used this design philosophy heavily on was the Dashboard page, which was broken into the header, sidebar, and documentCard. In these individual classes, components were further broken down by their tasks within the application. This also helped make our application more modular, because if any one component broke, it would likely not affect the others. Finally, from the beginning, we knew we wanted mobile compatibility for our application. During the creation of our prototype, we found that implementing mobile compatibility from a web-based platform can be annoying. After searching for some good practices on Google, we found that it is recommended to create the mobile implementation first, then expand to desktop. This greatly helped provide more prepared design choices that work on both mobile and desktop. This philosophy also helped create a better experience across different desktop screen sizes and a more enjoyable graphical interface overall.

Deployment

To deploy the WebToSpeech project, you begin by downloading the “webtospeech” folder currently available on the WebToSpeech GitHub repository, containing all of the React components, pages, scripting, etc. With Node.js installed, run “npm install” in the command line while within the directory to install any dependencies for the project.

The database is created by creating a new project in Subabase. From there, the corresponding tables need to be created either from the SQL files in the repository's document section or manually. The project has a ‘supabaseClient.js’ file located at the bottom of the src directory. Make sure the file is imported, then use it to create an instance of the supabaseClient using the Supabase URL and Supabase Anon Key. These can be found in the API settings of your Supabase project. Note that these keys should never be anywhere in the program other than the .env.local file, and that file should be added to the .gitignore file. Those variables should be used through reference. After setting up the variables in the .env.local file, they should be added as environment variables in the Vercel server. Finally, buckets will need to be made for both community books and user books. The community books bucket can be public. The user's books bucket should not be public, and use an RSL policy to create directories within the bucket that correspond to the user's UUID, limiting their access to those directories.

Google OAuth needs to be configured to connect a Google account to Google Cloud (Google Auth Platform) for the project, using the tokens generated in Supabase

and Google Auth. After that, Google Auth should be enabled and manual login disabled, so Google is the only login method.

With that, the project is now ready to be hosted locally by running “npm start” within the “webtospeech” directory. To host the project on a server, create a GitHub repository for it. On Vercel, choose “Import Git Repository”, and use your GitHub account to deploy the repository on a Vercel server. In Vercel, you’ll need to set the Supabase API key by going to settings > environment variables and creating two environment variables. The first will be called REACT_APP_SUPABASE_ANON_KEY and will have the key as the value. The second will be called REACT_APP_SUPABASE_URL and will have the Supabase URL as the value.

Known Issues

The search filtering feature is currently unimplemented. Clicking the button has no effect, and there is no way to filter searches other than by document name at the moment. We plan to add filtering options for author and genre, which we can implement in two ways. We can either handle filtering client-side with JavaScript to display only documents that match the search, or we can modify the query sent to Supabase to retrieve only documents that match the selected filter option.

Various text-to-speech control options are unimplemented. At the moment, the user can only use the default text-to-speech option provided by their browser. The user also has no way to control the position in the text where the dialogue is being read. To implement voice-changing, a button would be placed at the bottom of the page alongside the other controls. Clicking that button would display a modal with a dropdown menu of options supported by their device and browser, allowing them to change the voice to whatever they prefer. To set the reading position, we would allow the user to select text in the document, then show a context menu option to start reading from that selection.

When using some uncommon browsers, such as Kiwi, the browser in which we found the issue, the user will encounter a Supabase error when attempting to access the website. This is because the browser does not trust the Google OAuth authentication prompt and blocks it from loading. For now, we plan to support usability across major browsers and don’t intend to fix this issue unless it affects either Chrome or Safari.

When loaded across multiple devices, the profile picture may fail to load because of how we load it from Supabase. At the moment, we take the Google profile picture URL from Supabase and load that into an image element, but this URL provided by Google is rate-limited. To avoid this issue, we need to store the profile images ourselves and serve them to users on the Dashboard and Profile pages. We already have some caching in place to mitigate this, but the issue persists.

iOS and Mac devices have some functionality issues when running the website. The most notable one that we've diagnosed is an issue with the speed slider on the Document page. The intention is for the volume slider to cap at 0.5x and 2.0x speed. On iOS and Mac, the slider seems to have a significantly higher upper bound. This allows Apple devices to set the speed up to 8x the default, making the slider less practical to use and offering no real benefit to users.

Another issue arises from extremely fast inputs of the play/pause button on the document page. Whenever the button is pressed, the text-to-speech reader has to be instantiated as an object in the script. When the button is pressed in rapid succession, seemingly due to mouse settings that trigger double-clicks on a single mouse press, the reader will continue reading the document page even though the button is visually treated as paused.